

HTML

What is HTML?

- Stands for Hypertext Markup Language.
- Most documents that appear on the World Wide Web were written in HTML.
- HTML is a markup language, not a programming language. In fact, the term HTML is an acronym that stands for Hypertext Markup Language.
- We can apply this markup language to your pages to display text, images, sound and movie files, and almost any other type of electronic information.
- We use the language to format documents and link them together, regardless of the type of computer with which the file was originally created.

HTML Elements

- An element consists of three basic parts: an opening tag, the element's content, and finally, a closing tag.

<p> - opening paragraph tag
Element Content - paragraph words
</p> - closing tag

- Every (web) page requires four critical elements: the html, head, title, and body elements.

1. <html> Element...</html>

- <html> begins and ends each and every web page.
- Its purpose is to encapsulate all the HTML code and describe the HTML document to the web browser.

<html></html>

2. <head> Element

- The <head> element is "next" as they say. As long as it falls somewhere between your <html> tag and your web page content (<body>).
- The head functions "behind the scenes." Tags placed within the head element are not directly displayed by web browsers.
- We will be placing the <title> element here.
- Other elements used for scripting (JavaScript) and formatting (CSS) will eventually be introduced and you will have to place them within your head element.

<html>
<head>
</head>
</html>

3. The <title> Element

- Place the <title> tag within the <head> element to title your page.
- The words you write between the opening and closing <title></title> tags will be displayed at the top of a viewer's browser.

```
<html><head><title>My WebPage!</title></head></html>
```

4. The <body> Element

- The <body> element is where all content is placed. (Paragraphs, pictures, tables, etc).
- The body element will encapsulate all of your webpage's viewable content.

```
<html>
<head><title>My WebPage!</title></head>
<body>
Hello World! All my content goes here!
</body>
</html>
```

HTML Tags

- A web browser reads an HTML document top to bottom, left to right.
- Each time the browser finds a tag, it is displayed accordingly (paragraphs look like paragraphs, tables look like tables, etc).
- Tags have 3 major parts: opening tag(s), content(s), and closing tag(s).
- Recall that a completed tag is termed an element.

1. Paragraph Tag <p>

- The <p> tag defines a paragraph. Using this tag places a blank line above and below the text of the paragraph.

```
<p>Avoid losing floppy disks with important school...</p>
<p>For instance, let's say you had a HUGE school...</p>
```

2. HTML - Headings 1:6

- A heading in HTML is just what we might expect, a title or subtitle.
- By placing text inside of <h1> (heading) tags, the text displays bold and the size of the text depends on the number of heading (1-6).
- Headings are numbered 1-6, with 1 being the largest heading and 6 being the smallest.

```
<html><body>
<h1>This is heading 1</h1><h2>This is heading 2</h2><h3>This is heading 3</h3>
<h4>This is heading 4</h4><h5>This is heading 5</h5><h6>This is heading 6</h6>
</body></html>
```

3. Line Breaks

- Line breaks are different than most of the tags we have seen so far. A line break ends the line you are currently on and resumes on the next line.

```
<p>Darshan<br />  
Computer<br /></p>
```

HTML Lists

- There are 3 different types of lists.
- A tag starts an ordered list, for unordered lists, and <dl> for definition lists.
 1. - unordered list; *bullets*
 2. - ordered list; *numbers*
 3. <dl> - definition list; *dictionary*

1. HTML Ordered Lists

- Use the tag to begin an ordered list. Place the (list item) tag between your opening and closing tags to create list items.
- Ordered simply means numbered, as the list below demonstrates.

```
<ol>  
<li>Find a Job</li>  
<li>Move Out</li>  
</ol>
```

- Start your ordered list on any number besides 1 using the start attribute.

```
<ol start="4" >  
<li>Buy Food</li>  
<li>Get a Degree</li>  
</ol>
```

- There are 4 other types of ordered lists. Instead of generic numbers you can replace them with Roman numerals or letters, both capital and lower-case. Use the type attribute to change the numbering.

```
<ol type="a">  
<ol type="A">  
<ol type="i">  
<ol type="I">  
</ol>
```

2. HTML Unordered Lists

- Create a bulleted list with the tag. The bullet itself comes in three subtypes: squares, discs, and circles.
- The default bullet displayed by most web browsers is the traditional full disc.

```
<ul>
<li>Milk</li>
<li>Chocolate</li>
</ul>
```

- There are 3 other types of unordered lists.

```
<ol type="square">
<ol type="disc">
<ol type="circle">
</ol>
```

3. HTML Definition Term Lists

- Make definition lists as seen in dictionaries using the <dl> tag. These lists displace the term word just above the definition itself for a unique look. It's wise to bold the terms to displace them further.
 - <dl> - defines the start of the list
 - <dt> - definition term
 - <dd> - defining definition

```
<dl>
<dt><b>Fromage</b></dt>
<dd>French word for cheese.</dd>
<dt><b>Voiture</b></dt>
<dd>French word for car.</dd>
</dl>
```

4. HTML Nested Lists

- You can also nest one list within another, so you could make an unordered list inside a

```
<html>
<ol>
    <li> Clear out garage</li>
    <ul>
        <li> Tomatoes</li>
    </ul>
    <li> repair fence </li>
</ol>
</html>
```

numbered one:

HTML - Formatting Elements

- Several tags exist to further amplify text elements. These formatting tags can make text bold, italic, sub/superscripted, and more.

3 - HTML

Tag	Description	Example
	The tag specifies bold text.	Bold Text
<i>	The <i> tag specifies italic text.	<i>Italic Text</i>
	The tag specifies emphasis text	Emphasized Text
<sup>	The <sup> tag defines superscript text. Superscript text appears half a character above the baseline. Superscript text can be used for footnotes, like WWW ^[1] .	This text contains ^{superscript} text.
<sub>	The <sub> tag defines subscript text. Subscript text appears half a character below the baseline. Subscript text can be used for chemical formulas, like H ₂ O.	An example of _{subscripted Text}
<tt>	The <tt> tag defines teletype text.	<tt>This text is teletype text.</tt>
<blink>	The <blink> tag is used for blinking the text.	<blink> blinking text tag</blink>

HTML Color Coding System - Color Names

There are 3 different methods to set color.

- We can set color using three methods.
 - a. Using color name

```
<body bgcolor="red">  
<font color="red">
```

- b. Using RGB(Red, Green, Blue) value

```
<body bgcolor="rgb(72,0,0)">  
<font color="rgb(72,0,0)">
```

- c. Using Hexadecimal value

```
<body bgcolor="#ffff00">  
<font color="#ffff00">
```

HTML - Font and Basefont

- The tag is used to add style, size, and color to the text on your site. Use the size, color, and face attributes to customize your fonts.
- Use a <basefont> tag to set all of your text to the same size, face, and color.

1. Font Size

- Set the size of your font with size. The range of accepted values is from 1(smallest) to 7(largest).The default size of a font is 3.

```
<p><font size="5">Here is a size 5 font</font></p>
```

2. Font Color

- Set the color of your font with color.

```
<font color="#990000">This text is hexcolor #990000</font><br />
<font color="red">This text is red</font>
```

3. Font Face

- Choose a different font face using any font you have installed.

```
<p><font face="Bookman Old Style, Book Antiqua, Garamond">This paragraph has
had its font...</font></p>
```

4. Basefont - Set a Solid Base

- With the basefont tag you will be able to set the default font for your web page.

HTML Code:

```
<html><body>
<basefont size="2" color="green"><p>This paragraph has had its font...</p>
</basefont>
</body></html>
```

HTML - Hypertext Reference (href) or Hyperlinks

- The href attribute defines reference that the link refers to. Basically this is where the user will be taken if they wish to click this link.
- Use the <a> tags to define the start and ending of an anchor.
- Decide what type of href attribute you need and place this attribute into the opening tag.
- The text you place between the opening and closing tags will be shown as the link on a page. Use the demonstration below as a reference.
- Hypertext references can be Internal, Local, or Global.
- **Internal** - Links to anchors on the current page
- **Local** - Links to other pages within your domain
- **Global** - Links to other domains outside of your site

```
Internal - href="#anchorname"
Local - href="../pics/picturefile.jpg"
Global - href=http://www.xyz.com/
```

```
<a href="http://www.google.com/" target="_blank">Google Home</a>
<a href="http://www.espn.com/" target="_blank">ESPN Home</a>
<a href="http://www.yahoo.com/" target="_blank">Yahoo Home</a>
```

Link Targets

- The target attribute defines whether to open the page in a separate window, or to open the link in the current browser window.

HTML Code:	
target="_blank"	Opens new page in a new browser window
target="_self"	Loads the new page in current window
target="_parent"	Loads new page into a frame that is superior to where the link lies
target="_top"	Loads new page into the current browser window, cancelling all frames

Anchors

- To link to sections of your existing page a name must be given to the anchor.
- In the example below, we've created a mini Table of Contents for this page.
- By placing blank anchors just after each heading, and naming them, we can then create reference links to those sections on this page as shown below.
- First, the headings of this page contain blank, named anchors. They look like this.

```
<h2>HTML Links and Anchors<a name="top"></a></h2>
<h2>HTML Text Links<a name="text"></a></h2>
<h2>HTML Email<a name="email"></a></h2>
```

- Now create the reference links, placing the # symbol followed by the name of the anchor in the href of the new link.

```
<a href="#top">Go to the Top</a>
<a href="#text">Learn about Text Links</a>
<a href="#email">Learn about Email Links</a>
```

HTML – Images

- Use the tag to place an image on your web page.

```
<imgsrc="sunset.gif" />
```

1. Image src

- Above we have defined the src attribute.
- Src stands for source, the source of the image or more appropriately, where the picture file is located.
- There are two ways to define the source of an image. First you may use a standard URL. (src=http://www.Xyz.com/pics/htmlT/sunset.gif) As your second choice, you may copy or upload the file onto your web server and access it locally using standard directory tree methods. (src="../sunset.gif")
- The location of this picture file is in relation to your location of your .html file.

URL Types:

Local Src	Location Description
src="sunset.gif"	picture file resides in same directory as .html file
src="../sunset.gif"	picture file resides in previous directory as .html file
src="../pics/sunset.gif"	picture file resides in the pic directory in a previous directory as .html file

- A URL cannot contain drive letters
- Therefore something like src="C:\\www\\web\\pics\\" will not work. Pictures must be uploaded along with your .html file to your web server.

2. Alternative Attribute

- The alt attribute specifies alternate text to be displayed if for some reason the browser cannot find the image, or if a user has image files disabled.

```
<imgsrc="http://example.com/brokenlink/sunset.gif" alt="Beautiful Sunset" />
```

3. Image Height and Width

- To define the height and width of the image, rather than letting the browser compute the size, use the height and width attributes.

```
<imgsrc="sunset.gif" height="50" width="100">
```

4. Vertically and Horizontally Align Images

- Use the align and valign attributes to place images within your body, tables, or sections.
 1. align (Horizontal)
 1. right 2. left 3. center
 2. valign (Vertical)
 1. top 2. bottom 3. center
- Below is an example of how to align an image to the right of a paragraph

```
<p>This is paragraph 1, yes it is...</p>
```

```
<p><imgsrc="sunset.gif" align="right">The image will appear along the...isn't it? </p>
```

5. Images as Links

- Images are very useful for links and can be created with the HTML below.

```
<a href="http://www.xyz.com/"><imgsrc="sunset.gif"></a>
```

HTML Forms

- A form will take input from the viewer and depending on your needs, you may store that data into a file, place an order, gather user statistics, register the person to your web forum, or maybe subscribe them to your weekly newsletter.

Making a Form

- `<form>` is main tag to build a form.
- It has a few optional attributes too. Below is an example of the form element.

```
<form action="processform.php" method="post">  
</form>
```

- The action attribute tells the HTML where to send the collected information, while the method attribute describes the way to send it.

Type of Input

- The main tag for collecting information from the user is `<input>`.
- The tag itself contains a name attribute, so that we can refer to the input by a name, and the size of the entry box in characters.
- There are quite a few different types of input to choose from:
- `<input type="text"/>` this is the default input type and accepts characters and numbers into a text box. It can also have a value attribute attached to it, which will give it an initial value.
- `<input type="password"/>` this is similar to the above text box but anything that is typed cannot be seen; instead an asterisk is printed to cover up the entry. As the name suggests, this is used for password entry.
- `<input type="checkbox"/>` this gives a box that can be toggled between checked and unchecked. It can initially be set to one or the other with `checked="checked"`.
- `<input type="radio"/>` this is similar to checkbox but in group of radio buttons only one can be selected at a time. This can also have an initial checked state on one of the radio buttons.
- `<input type="file"/>` This will give a box to allow you to choose a file similar to when you open or save files usually on your machine. It can be used to select a file on the local machine for upload to a server.

3 - HTML

- `<input type="submit"/>` this allows a form to be submitted. When pressed, the information will be passed on for processing, usually to a script mentioned in the action attribute option of the form.
- `<input type="image"/>` this will also submit the form when selected and, like the `img` tag, requires the `src` attribute to specify an associated image.
- `<input type="button"/>` this makes a button available.
- `<input type="reset"/>` this will reset the form to its initial state when selected.
- `<input type="hidden"/>` this allows hidden data(not seen by the user) to be passed along with the form.

HTML Text Fields

- The `<input>` has a few attributes that you should be aware of.
- **type** - Determines what kind of input field it will be. Possible choices are text, submit, and password.
- **name** - Assigns a name to the given field so that you may reference it later.
- **size** - Sets the horizontal width of the field. The unit of measurement is in blank spaces.
- **maxlength** - Dictates the maximum number of characters that can be entered.

```
<form method="post" action="mailto:youremail@email.com">  
Name: <input type="text" size="10" maxlength="40" name="name"><br />  
Password: <input type="password" size="10" maxlength="10"  
name="password">
```

HTML Radio Buttons

- Radio buttons are a popular form of interaction. You may have seen them on quizzes, questionnaires, and other web sites that give the user a multiple choice question. that relate to the radio button.

```
<form method="post" action="mailto:youremail@email.com">  
What kind of shirt are you wearing? <br />  
Shade:  
<input type="radio" name="shade" value="dark">Dark  
<input type="radio" name="shade" value="light">Light <br />  
</form>
```

HTML Check Boxes

- Check boxes allow for multiple items to be selected for a certain group of choices. The check box's name and value attributes behave the same as a radio button.

```
<form method="post" action="mailto:youremail@email.com">  
Select your favorite cartoon characters.  
<input type="checkbox" name="toon" value="Goofy">Goofy  
<input type="checkbox" name="toon" value="Donald">Donald  
<input type="checkbox" name="toon" value="Bugs">Bugs Bunny  
</form>
```

HTML Drop Down Lists

- Drop down menus are created with the <select> and <option> tags. <select> is the list itself and each <option> is an available choice for the user.

```
<form method="post" action="mailto:youremail@email.com">  
College Degree?  
<select name="degree">  
<option>Choose One</option>  
<option>Some High School</option>  
<option>High School Degree</option>  
</select>  
</form>
```

HTML Selection List

- Yet another type of form, a highlighted selection list. This form will post what the user highlights. Basically just another type of way to get input from the user.
- The size attribute selects how many options will be shown at once before needing to scroll, and the selected option tells the browser which choice to select by default.

```
<form method="post" action="mailto:youremail@email.com">  
Musical Taste  
<select multiple name="music" size="4">  
<option value="emo" selected>Emo</option>  
<option value="metal/rock">Metal/Rock</option>  
<option value="hiphop">Hip Hop</option><option value="ska">Ska</option>  
<option value="jazz">Jazz</option>  
</form>
```

HTML Text Areas

- Text areas serve as an input field for viewers to place their own comments onto forums and the like use text areas to post what you type onto their site using scripts. For this form, the text area is used as a way to write comments to somebody.
- Rows and columns need to be specified as attributes to the <textarea> tag.

- Another attribute to be aware of is the wrap. Wrap has 3 values.

```
<form method="post" action="mailto:youremail@email.com">
<textarea rows="5" cols="20" name="comments"> Enter Comments Here
</textarea>
</form>
```

HTML Tables

- The <table> tag is used to begin a table. Within a table element are the <tr> (table rows) and <td> (table columns) tags.

```
<table border="1">
<tr><td>Row 1 Cell 1</td><td>Row 1 Cell 2</td></tr>
<tr><td>Row 2 Cell 1</td><td>Row 2 Cell 2</td></tr>
</table>
```

Row 1 Cell 1	Row 1 Cell 2
Row 2 Cell 1	Row 2 Cell 2

- Content is placed within tables cells. A table cell is defined by <td> and </td>. The border attribute defines how wide the table's border will be.

Spanning Multiple Rows and Cells

- Use rowspan to span multiple rows and colspan to span multiple columns.
- Note: if you would like to place headers at the top of your columns, use the <th> tag as shown below. By default these headers are bold to set them apart from the rest of your table's content.

```
<table border="1"><tr><th>Column 1</th><th>Column 2</th><th>Column 3</th></tr>
<tr><td rowspan="2">Row 1 Cell 1</td><td>Row 1 Cell 2</td><td>Row 1 Cell 3</td></tr>
<tr><td>Row 2 Cell 2</td><td>Row 2 Cell 3</td></tr>
<tr><td colspan="3">Row 3 Cell 1</td></tr>
</table>
```

Column 1	Column 2	Column 3
Row 1 Cell 1	Row 1 Cell 2	Row 1 Cell 3
	Row 2 Cell 2	Row 2 Cell 3
Row 3 Cell 1		

Cell Padding and Spacing

- With the cellpadding and cellspacing attributes you will be able to adjust the white space on your tables. Spacing defines the width of the border, while padding represents the distance between cell borders and the content within. Color has been added to the table to emphasize these attributes.

3 - HTML

```
<table border="1" cellspacing="10" bgcolor="rgb(0,255,0)">
<tr><th>Column 1</th><th>Column 2</th></tr>
<tr><td>Row 1 Cell 1</td><td>Row 1 Cell 2</td></tr>
<tr><td>Row 2 Cell 1</td><td>Row 2 Cell 2</td></tr>
</table>
```

Column 1	Column 2
Row 1 Cell 1	Row 1 Cell 2
Row 2 Cell 1	Row 2 Cell 2

- And now we will change the cellpadding of the table and remove the cellspacing from the previous example.

```
<table border="1" cellpadding="10" bgcolor="rgb(0,255,0)">
<tr><th>Column 1</th><th>Column 2</th></tr>
<tr><td>Row 1 Cell 1</td><td>Row 1 Cell 2</td></tr>
<tr><td>Row 2 Cell 1</td><td>Row 2 Cell 2</td></tr>
</table>
```

Column 1	Column 2
Row 1 Cell 1	Row 1 Cell 2
Row 2 Cell 1	Row 2 Cell 2

HTML - <!-- Comments -->

- A comment is a way for you as the web page developer to control what lines of code are to be ignored by the web browser.
- Comment syntax may be a little complicated, there is an opening and a closing much like tags.
 1. <!-- Opening Comment
 2. --> Closing Comment

```
<!--Note to self: This is my banner image! Don't forget -->
<imgsrc="http://www.website.com/pics/anyimage.jpg" height="100" width="200"/>
```

What is an HTML Form? Discuss different form attributes

- Form is a data collection mechanism within HTML that allows the design of various styles of input to suit most types of information.
- An input element can vary in many ways, depending on the type attribute. An input element can be of type textfield, checkbox, password, radiobutton, submit button, and more.
- Following are **attributes of <form>**.

1. Name:

- The name attribute specifies the name of a form which is used to reference elements in a JavaScript.

`<form action="URL">` Value : URL
Description : Where to send the form data.

2. Action:

- The required action attribute specifies where to send the form-data when a form is submitted.

`<form action="URL">` Value : URL
Description : Where to send the form data.

3. Method :

- The method attribute specifies how to send form-data (the form-data is sent to the page specified in the action attribute).

`<form method="get|post">`
Value : get
Description : Default. Appends the form-data to the URL in name/value pairs:
URL?name=value&name=value
Value : post
Description : Sends the form-data as an HTTP post transaction.

4. Target

- The target attribute specifies a name or a keyword that indicates where to display the response that is received after submitting the form.

`<form target="_blank|_self|_parent|_top|frameName">`

Value	Description
_blank	The response is displayed in a new window or tab
_self	The response is displayed in the same frame (this is default)
_parent	The response is displayed in the parent frame
_top	The response is displayed in the full body of the window
frameName	The response is displayed in a named frame

Explain following terms with example.

(1) <optgroup> :

- The <optgroup> is used to group related options in a drop-down list. If you have a long list of options, groups of related options are easier to handle for a user.

```
<select>
  <optgroup label="Swedish Cars">
    <option value="volvo">Volvo</option>
    <option value="saab">Saab</option>
  </optgroup>
  <optgroup label="German Cars">
    <option value="mercedes">Mercedes</option>
    <option value="audi">Audi</option>
  </optgroup>
</select>
```

(2) :

- The tag is used to group inline-elements in a document.
- The tag provides no visual change by itself.
- The tag provides a way to add a hook to a part of a text or a part of a document.
- When the text is hooked in a element you can add styles to the content, or manipulate the content with for example JavaScript.

```
<p>My mother has <span style="color:lightblue">lightblue</span> eyes.</p>
```

Introduction to HTML5

- The DOCTYPE declaration for HTML5 is very simple:
`<!DOCTYPE html>`
- The character encoding (charset) declaration is also very simple:
`<meta charset="UTF-8">`
- New HTML5 Elements:
 - New semantic elements like `<header>`, `<footer>`, `<article>`, and `<section>`.
 - New form control attributes like number, date, time, calendar, and range.
 - New graphic elements: `<svg>` and `<canvas>`.
 - New multimedia elements: `<audio>` and `<video>`.
 - Elements Removed in HTML5

The following HTML4 elements have been removed from HTML5:

Element	Use instead
<code><acronym></code>	<code><abbr></code>
<code><applet></code>	<code><object></code>
<code><basefont></code>	CSS
<code><big></code>	CSS
<code><center></code>	CSS
<code><dir></code>	<code></code>
<code></code>	CSS
<code><frame></code>	
<code><frameset></code>	
<code><noframes></code>	
<code><strike></code>	CSS
<code><tt></code>	CSS

What is CSS?

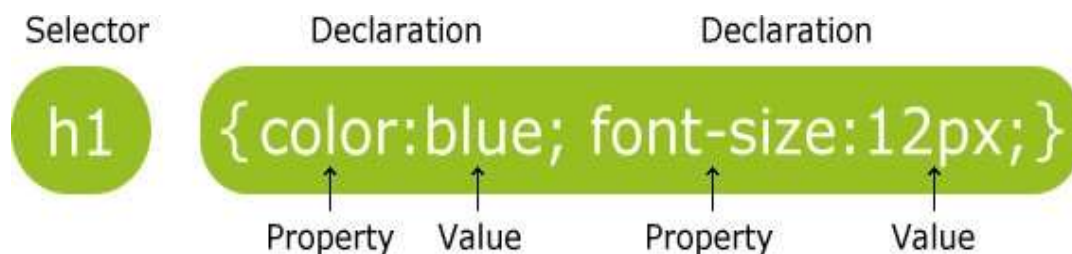
- CSS stands for Cascading Style Sheets
 - Styles define how to display HTML elements
 - External Style Sheets can save a lot of work
 - External Style Sheets are stored in CSS files
-

Importance of CSS

- CSS defines HOW HTML elements are to be displayed.
 - Styles are normally saved in external .css files. External style sheets enable you to change the appearance and layout of all the pages in a Web site, just by editing one single file.
-

CSS Syntax

- A CSS rule has two main parts: a selector, and one or more declarations:



- The selector is normally the HTML element you want to style.
 - Each declaration consists of a property and a value.
 - The property is the style attribute you want to change. Each property has a value.
-

What is the difference between class and id?

The id Selector

- The id selector is used to specify a style for a single, unique element.
- The id selector uses the id attribute of the HTML element, and is defined with a "#".
- The style rule below will be applied to the element with id="para1":

```
#para1  
{ text-align:center; color:red; }
```

The class Selector

- The class selector is used to specify a style for a group of elements. Unlike the id selector, the class selector is most often used on several elements.
- This allows you to set a particular style for many HTML elements with the same class.
- The class selector uses the HTML class attribute, and is defined with a "."
- In the example below, all HTML elements with class="center" will be center-aligned:

```
.center {text-align:center;}
```

- We can use more than one class in a single element

```
<a class="Center bold italic">
```

Explain different ways to write the CSS. / Explain CSS with all types. / Enlist and explain methods of using CSS in web page.

- There are three ways of inserting a style sheet:
 - External style sheet
 - Internal/Embedded style sheet
 - Inline style

1. External Style Sheet

- When using CSS it is preferable to keep the CSS separate from your HTML.
- Placing CSS in a separate file allows the web designer to completely differentiate between content (HTML) and design (CSS).
- External CSS is a file that contains only CSS code and is saved with a ".css" file extension.
- This CSS file is then referenced in your HTML using the <link> instead of <style>.

File Creation

- Open up notepad.exe, or any other plain text editor and type the following CSS code.

```
body{ background-color: gray;} p { color: blue; }h3{ color: white; }
```

- Save the file as a CSS (.css) file.
- Name the file "test.css" (without the quotes). Now create a new HTML file and fill it with the following code.

```
<html><head>
<link rel="stylesheet" type="text/css" href="test.css" /></head>
<body>
<h3> A White Header </h3>
<p> This paragraph has a blue font.
The background color of this page is gray because we changed it with CSS! </p>
</body></html>
```

Why Use External CSS?

- It keeps your website design and content separate.

- It's much easier to reuse your CSS code if you have it in a separate file. Instead of typing the same CSS code on every web page you have, simply have many pages refer to a single CSS file with the "link" tag.
- You can make drastic changes to your web pages with just a few changes in a single CSS file.

2. Internal/Embedded CSS

- This type of CSS is only for Single Page.
- When using internal CSS, we must add a new tag, <style>, inside the <head> tag. The HTML code below contains an example of <style>'s usage.

```
<html><head>
<style type="text/css"></style>
</head><body>
<p>Your page's content!</p></body>
</html>
```

Creating Internal CSS Code

- Below is an example of simple CSS code.

```
<html><head>
<style type="text/css">
p {color: white; }
body {background-color: black; }
</style></head><body>
<p>White text on a black background!</p></body>
</html>
```

3. Inline CSS

- It is possible to place CSS right in your HTML code, and this method of CSS usage is referred to as inline css.
- Inline CSS has the highest priority out of external, internal, and inline CSS.
- This means that you can override styles that are defined in external or internal by using inline CSS.
- If you want to add a style inside an HTML element all you have to do is specify the desired CSS properties with the style HTML attribute.

```
<html><head>
<link rel="stylesheet" type="text/css" href="test.css" /></head>
<body>
<p style="background: blue; color: white;">A new background and font color with
inline CSS</p></body>
</html>
```

Explain CSS Background with all its attributes

- CSS background properties are used to define the background effects of an element.

1. CSS Background Color

- The background-color property specifies the background color of an element.

- The background color of a page is defined in the body selector:
- Below is example of CSS backgrounds

```
body {background-color:#b0c4de;}
```

2. CSS Background Image

- The background-image property specifies an image to use as the background of an element.

```
body {background-image:url('paper.gif');}
```

3. Background Image Repeat

- You can have a background image repeat vertically (y-axis), horizontally (x-axis), in both directions, or in neither direction.

```
p {background-image: url(smallPic.jpg); background-repeat: repeat; }
h4 {background-image: url(smallPic.jpg); background-repeat: repeat-y; }
ol {background-image: url(smallPic.jpg); background-repeat: repeat-x; }
ul {background-image: url(smallPic.jpg);background-repeat: no-repeat; }
```

4. CSS Fixed Background Image

- The background-attachment property sets whether a background image is fixed or scrolls with the rest of the page.

```
textarea.noScroll { background-image: url(smallPic.jpg); background-attachment: fixed;}
textarea {
background-image: url(smallPic.jpg);
background-attachment: scroll; }
```

5. CSS Background Image Positioning

- The background-position property sets the starting position of a background image.

```
p {background-image: url(smallPic.jpg); background-position: 20px 10px;}
h4 {background-image: url(smallPic.jpg); background-position: 30% 30%;}
ol {background-image: url(smallPic.jpg); background-position: top center;}
```

Explain CSS Font with all its attributes

- CSS font properties define the font family, boldness, size, and the style of a text.

1. CSS Font Color

- Set the text-color for different elements:

```
h4 { color: red; }
h5 { color: #9000A1; }
h6 { color: rgb(0, 220, 98); }
```

2. CSS Font Family

- The font family of a text is set with the font-family property.

```
h4 { font-family: sans-serif; }h5 { font-family: serif; }
h6 { font-family: arial; }
```


3. CSS Font Size

- The font-size property sets the size of the text.

```
p { font-size: 120%; } ol { font-size: 10px; } ul { font-size: x-large; }
```

4. CSS Font Style

- The font-style property is mostly used to specify italic text.
- This property has three values:
 - normal - The text is shown normally
 - italic - The text is shown in italics
 - oblique - The text is "leaning" (oblique is very similar to italic, but less supported)

```
p { font-style: italic; } h4 { font-style: oblique; }
```

5. CSS Font Weight

- The font-weight property sets how thick or thin characters in text should be displayed.

```
p { font-weight: 100; } ul { font-weight: bolder; }
```

6. CSS Font Variant

- The font-variant property specifies whether or not a text should be displayed in a small-caps font.

```
p { font-variant: small-caps; }
```

Explain CSS Text with all its attributes.

- While CSS Font covers most of the traditional ways to format your text, CSS Text allows you to control the spacing, decoration, and alignment of your text.

1. Text Decoration

- The text-decoration property is used to set or remove decorations from text.
- The text-decoration property is mostly used to remove underlines from links for design purposes.

```
h4 { text-decoration: line-through; }
h5 { text-decoration: overline; }
h6 { text-decoration: underline; }
a { text-decoration: none; }
```

2. Text Indent

- The text-indentation property is used to specify the indentation of the first line of a text.

```
p { text-indent: 20px; } h5 { text-indent: 30%; }
```

3. Text Align

- The text-align property is used to set the horizontal alignment of a text.

```
p { text-align: right; }
h5 { text-align: justify; }
```

4. Text Transform

- The text-transform property is used to specify uppercase and lowercase letters in a text.

```
p { text-transform: capitalize; } h5 { text-transform: uppercase; }
```

5. CSS White Space

- The white-space attribute allows you to prevent text from wrapping until you place a break `<br />` into your text.

```
p { white-space: nowrap; }
```

6. CSS Word Spacing

- With the CSS attribute word-spacing you are able to specify the exact value of the spacing between your words. Word-spacing should be defined with exact values.

```
p { word-spacing: 10px; }
```

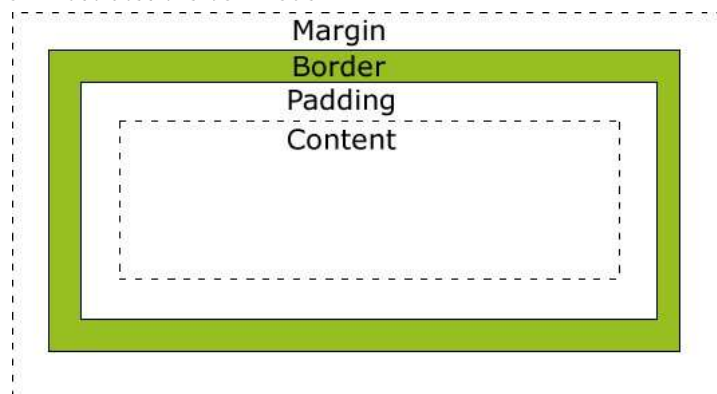
7. CSS Letter Spacing

- With the CSS attribute letter-spacing you are able to specify the exact value of the spacing between your letters. Letter-spacing should be defined with exact values.

```
p { letter-spacing: 3px; }
```

Explain BOX MODEL.

- All HTML elements can be considered as boxes. In CSS, the term "box model" is used when talking about design and layout.
- The CSS box model is essentially a box that wraps around HTML elements, and it consists of: margins, borders, padding, and the actual content.
- The box model allows us to place a border around elements and space elements in relation to other elements.
- The image below illustrates the box model:



- Explanation of the different parts:
 - **Margin** - Clears an area around the border. The margin does not have a background color, it is completely transparent
 - **Border** - A border that goes around the padding and content. The border is affected by the background color of the box
 - **Padding** - Clears an area around the content. The padding is affected by the background color of the box
 - **Content** - The content of the box, where text and images appear

Explain CSS Padding.

- The CSS padding properties define the space between the element border and the element content.

p {padding: 15px; border: 1px solid black;}

- The top, right, bottom, and left padding can be changed independently using separate properties. A shorthand padding property can also be used, to change all paddings at once.

1. Possible Values

Value	Descriptions
length	Defines a fixed padding (in pixels, pt, em, etc.)
%	Defines a padding in % of the containing element.

padding-top:25px;

padding-bottom:25px;

padding-right:50px;

padding-left:50px;

2. Padding - Shorthand property

- To shorten the code, it is possible to specify all the padding properties in one property. This is called a shorthand property.

padding:25px 50px;

Explain CSS Margin.

- The CSS margin properties define the space around elements.

p {margin: 5px; border: 1px solid black; }

- The top, right, bottom, and left margin can be changed independently using separate properties. A shorthand margin property can also be used, to change all margins at once.

Value	Descriptions
auto	The browser calculates a margin
length	Specifies a margin in px, pt, cm, etc. Default value is 0px
%	Specifies a margin in percent of the width of the containing element
inherit	Specifies that the margin should be inherited from the parent element

1. Margin - Individual sides

- In CSS, it is possible to specify different margins for different sides:

margin-top:100px;

margin-bottom:100px;

margin-right:50px;

margin-left:50px;

2. Margin - Shorthand property

- To shorten the code, it is possible to specify all the margin properties in one property. This is called a shorthand property.

```
margin:100px 50px;
```

Explain CSS Border with all its attributes.

- The CSS border properties allow you to specify the style and color of an element's border.

1. Border Style Types

- The border-style property specifies what kind of border to display.

```
p.solid {border-style: solid; } p.double {border-style: double; } p.groove {border-style: groove; }
p.dotted {border-style: dotted; } p.dashed {border-style: dashed; } p.inset {border-style: inset; }
p.outset {border-style: outset; } p.ridge {border-style: ridge; } p.hidden {border-style: hidden; }
```

2. Border Width

- The border-width property is used to set the width of the border.

```
table { border-width: 7px; border-style: outset; }
td { border-width: medium; border-style: outset; }
p { border-width: thick; border-style: solid; }
```

3. Border Color

- The border-color property is used to set the color of the border.
- Border colors can be any color defined by RGB, hexadecimal, or key terms. Below is an example of each of these types.

```
table { border-color: rgb( 100, 100, 255); border-style: dashed; }
td { border-color: #FFBD32; border-style: ridge; }
p { border-color: blue; border-style: solid; }
```

4. Border: border-(direction)

- If you would like to place a border on only one side of an HTML element, or maybe have a unique look for each side of the border, then use border-(direction).
- The direction choices are of course: top, right, bottom, and left. CSS allows you to treat each side of a border separately from the other three sides.
- Each side can have its own color, width, and style set, as shown below.

```
p { border-bottom-style: dashed ; border-bottom-color: yellow; border-bottom-width: 5px; }
h4 { border-top-style: double; border-top-color: purple; border-top-width: thick; }
```

Explain CSS Lists with all its attributes.

- The CSS list properties allow you to:
 - Set different list item markers for ordered lists
 - Set different list item markers for unordered lists
 - Set an image as the list item marker

1. CSS List Style Type

- Specify all the list properties in one declaration.
 - Unordered list styles: square, circle, disc (default), and none
 - Ordered list styles: upper-alpha, lower-alpha, upper-roman, lower-roman, decimal (default), and none

```
ol { list-style-type: upper-roman; }
ul { list-style-type: circle; }
```

2. CSS Lists with Images

- Specify an image as the list-item marker in a list:

```
ul { list-style-image: url("listArrow.gif"); }
ol { list-style-image: url("listArrow2.gif"); }
```

3. CSS List Position

- With Specify that the the list-item markers should appear inside the content flow (results in an extra indentation)

```
ul { list-style-position: inside; }
ol { list-style-position: outside; }
```

- **Note:** "Outside" is actually the default setting for indentation.

Explain CSS Links

1. CSS Anchor/Link States

- The four links states are:
 - a:link - a normal, unvisited link
 - a:visited - a link the user has visited
 - a:hover - a link when the user mouse over it
 - a:active - a link the moment it is clicked

```
a:link{color:#FF0000;} /*unvisited link*/
a:visited{color:#00FF00;} /* visited link */
a:hover{color:#FF00FF;} /* mouse over link */
a:active {color:#0000FF;} /* selected link */
```

2. Text Decoration

- The text-decoration property is mostly used to remove underlines from links.

```
a:link {text-decoration:none;}
a:visited {text-decoration:none;}
a:hover {text-decoration:underline;}
a:active {text-decoration:underline;}
```

3. Background Color

- The background-color property specifies the background color for links.

```
a:link {background-color:#B2FF99;}
a:visited {background-color:#FFFF85;}
a:hover {background-color:#FF704D;}
a:active {background-color:#FF704D;}
```

Explain CSS Position with example.

- With the knowledge of CSS Positioning we will be able to manipulate the exact position of your HTML elements.

1. Position Relative

- Relative positioning changes the position of the HTML element relative to where it normally appears.
- If we had a header that appears at the top of our page, we could use relative positioning to move it a bit to the right and down a couple of pixels. Below is an example.

```
h3 {position: relative; top: 15px;left: 150px;}
p {position: relative; left: -10px;}
```

2. Position Absolute

- With absolute positioning, you define the exact pixel value where the specified HTML element will appear.
- The point of origin is the top-left of the browser's viewable area, so be sure you are measuring from that point.

```
h3 {position: absolute; top: 50px;left: 45px;}
p {position: absolute; top: 75px;left: 75px;}
```

Explain CSS Layers. / z-index property

- CSS allows you to control which item will appear on top with the use of layers.
- In CSS, each element is given a priority.
- If there are two overlapping CSS positioned elements, the element with the higher priority will appear on top of the other.
- To manually define a priority, set the z-index value. The larger the value, the higher the priority the element will have.

```
h4 {position: relative; top: 30px;left: 50px; z-index: 2;}
p {position: relative; z-index: 1;background-color: #FFCCCC;}
```

- This paragraph has a z-index of 1, which is less than the header.
 - If we had not defined the z-index, by default the paragraph would have been on top of the header because it appears later in our HTML code.
-

Explain CSS Float property.

- With CSS float, an element can be pushed to the left or right, allowing other elements to wrap around it.
- Wrapping text around an image is easy when using the CSS Float attribute.
- You have a choice to either float the picture to the left or to the right and the rest is done for you.

```
img.floatLeft { float: left; margin: 4px;}
img.floatRight { float: right; margin: 4px;}
```

```
<body>
<p>The images are contained with...</p>
<p>This second paragraph has an...</p>
</body>
```

Introduction to CSS3

- CSS3 is the latest standard for CSS.
- CSS3 is completely backwards-compatible with earlier versions of CSS.
- CSS3 has been split into "modules". It contains the "old CSS specification" (which has been split into smaller pieces). In addition, new modules are added.
- CSS3 Transitions are a presentational effect which allow property changes in CSS values, such as those that may be defined to occur on :hover or :focus, to occur smoothly over a specified duration – rather than happening instantaneously as is the normal behaviour.
- Transition effects can be applied to a wide variety of CSS properties, including background-color, width, height, opacity, and many more.
- Some of the most important CSS3 modules are:
 - Selectors
 - Box Model
 - Backgrounds and Borders
 - Image Values and Replaced Content
 - Text Effects
 - 2D/3D Transformations
 - Animations
 - Multiple Column Layout
 - User Interface

What is JavaScript?

- HTML and CSS concentrate on a static rendering of a page; things do not change on the page over time, or because of events.
 - To do these things, we use scripting languages, which allow content to change dynamically.
 - Not only this, but it is possible to interact with the user beyond what is possible with HTML.
 - Scripts are programs just like any other programming language; they can execute on the client side or the server.
-

Advantages of client side scripting

- The web browser uses its own resources, and eases the burden on the server.
 - It has fewer features than server side scripting.
 - It saves network bandwidth.
-

Disadvantages of client side scripting

- Code is usually visible.
 - Code is probably modifiable.
 - Local files and databases cannot be accessed.
 - User is able to disable client side scripting.
-

Differentiate between server side and client side scripting languages

Client-side scripting languages

- The client-side environment used to run scripts is usually a browser.
- The processing takes place on the end users computer.
- The source code is transferred from the web server to the user's computer over the internet and run directly in the browser.
- The scripting language needs to be enabled on the client computer.
- Sometimes if a user is conscious of security risks they may switch the scripting facility off.
- When this is the case a message usually pops up to alert the user when script is attempting to run.

Server-side scripting languages

- The server-side environment that runs a scripting language is a web server.
- A user's request is fulfilled by running a script directly on the web server to generate dynamic HTML pages.
- This HTML is then sent to the client browser.
- It is usually used to provide interactive web sites that interface to databases or other data stores on the server.

5 - JavaScript

- This is different from client-side scripting where scripts are run by the viewing web browser, usually in JavaScript.
 - The primary advantage to server-side scripting is the ability to highly customize the response based on the user's requirements, access rights, or queries into data stores.
-

What is difference between Java script and JAVA?

- Java is a statically typed language; JavaScript is dynamic.
 - Java is class-based; JavaScript is prototype-based.
 - Java constructors are special functions that can only be called at object creation; JavaScript "constructors" are just standard functions.
 - Java requires all non-block statements to end with a semicolon; JavaScript inserts semicolons at the ends of certain lines.
 - Java uses block-based scoping; JavaScript uses function-based scoping.
 - Java has an implicit this scope for non-static methods, and implicit class scope; JavaScript has implicit global scope.
-

Embedded JavaScript

- JavaScript can be embedded in an HTML document.
- To embed it in HTML you must write:

```
<script type="text/javascript">  
</script>
```
- The script tag has effect of the stopping the JavaScript being printed out as well as indentifying the code enclosed.
- The JavaScript can be placed in the head section of your HTML or the body.

```
<html>  
<body>  
<script type="text/javascript">  
    document.write("<h1>This is a heading</h1>");  
</script>  
</body>  
</html>
```

- The Scripts placed in the body section are executed as the page loads and can be used to generate the content of the page.
- As well as the body section, JavaScript can also be placed in the head part.
- The advantages of putting a script in there are that it loads before the main body.

External JavaScript

- If you want to use the same script on several pages it could be a good idea to place the code in a separate file, rather than writing it on each.
- That way if you want to update the code, or change it, you only need to do it once.

5 - JavaScript

- Simply take the code you want in a separate file out of your program and save it with the extension .js.

```
<html>
<body>
<script src="myScript.js"></script>
</body>
</html>
```

JavaScript Variables

- Variables in JavaScript behave the same as variables in most popular programming languages (C, C++, etc) do, but in JavaScript you don't have to declare variables before you use them.
- A variable's purpose is to store information so that it can be used later. A variable is a symbolic name that represents some data that you set.
- When using a variable for the first time it is not necessary to use "**var**" before the variable name.
- Variable names must begin with a letter.
- Variable names are case sensitive (y and Y are different variables).

```
var x=5;
var y=6;
var z=x+y;
```

- You can declare many variables in one statement. Just start the statement with var and separate the variables by comma:

```
var name="Doe", age=30, job="carpenter";
var name="Doe",
age=30,
job="carpenter";
```

- Variable declared without a value will have the value **undefined**.
- If you re-declare a JavaScript variable, it will not lose its value.
- The value of the variable *carname* will still have the value "Volvo" after the execution of the following two statements.

```
varcarname="Volvo";
varcarname;
```

JavaScript Operators

- Operators in JavaScript are very similar to operators that appear in other programming languages.
- The definition of an operator is a symbol that is used to perform an operation.
- Most often these operations are arithmetic (addition, subtraction, etc), but not always.

5 - JavaScript

Operator	Name
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus
=	Assignment

```
<body>
<script type="text/JavaScript">
<!--
var two = 2
var ten = 10
varlinebreak = "<br />"
document.write("two plus ten = ")
var result = two + ten
document.write(result)
//-->
</script>
</body>
```

Assignment	Equivalent to
X+=Y	X=X+Y
X-=Y	X=X-Y
X*=Y	X=X*Y
X/=Y	X=X/Y
X%=Y	X=X%Y

JavaScript Array

- An array is a special variable, which can hold more than one value at a time.
- The Array object is used to store multiple values in a single variable.
- An array can be created in three ways.
- The following code creates an Array object called myCars.

1. Regular

```
varmyCars=new Array();
myCars[0]="Saab";
myCars[1]="Volvo";
myCars[2]="BMW";
```

2. Condensed

```
varmyCars=new Array("Saab","Volvo","BMW");
```

3. Literal

```
var myCars=["Saab","Volvo","BMW"];
```

Access an Array

- You refer to an element in an array by referring to the **index** number.
- This statement access the value of the first element in myCars.

```
var name=myCars[0];
```

- This statement modifies the first element in myCars:

```
myCars[0]="Opel";
```

JavaScript Functions

- A function is a section of code that is separate from the main program.
- It is defined once but can be invoked many times.
- A function can be passed as parameters so that they can be used and a value can be returned back.
- There are some functions already built in to JavaScript, such as the Math.cos() function, which calculates the cosine of an angle.
- An example function could be:

```
function multByTen(x)
{
    return x*10;
}
```

- This can then be invoked by using the function's name complete with any parameters you want to pass:

```
mySum=multByTen(3)
```

- Below is an example of JavaScript function.

```
<html><body>
<script type="text/javascript">
var z= multXbyY(10,15);
document.write("The result is" +z);
function multXbyY(x,y) {
    document.write("x is " +x);
    document.write("y is " +y);
    return x*y;
}
</script>
</body></html>
```

JavaScript Conditions

- Conditional statements are used to perform different actions based on different conditions.
- In JavaScript we have the following conditional statements:
 - **if statement** - use this statement to execute some code only if a specified condition is true
 - **if...else statement** - use this statement to execute some code if the condition is true and another code if the condition is false
 - **if...else if ...else statement** - use this statement to select one of many blocks of code to be executed
 - **switch statement** - use this statement to select one of many blocks of code to be executed

If Statement

- Use the if statement to execute some code only if a specified condition is true.

```
if (condition)
{
    code to be executed if condition is true
}
```

If...else Statement

- Use the if....else statement to execute some code if a condition is true and another code if the condition is not true.

```
if (condition)
{
    code to be executed if condition is true
}
else
{
    code to be executed if condition is not true
}
```

If...else if...else Statement

- Use the if....else if...else statement to select one of several blocks of code to be executed.

```
if (condition1) {
    code to be executed if condition1 is true
}
else if (condition2) {
    code to be executed if condition2 is true
}
else {
    code to be executed if neither condition1 nor condition2 is true
}
```

Switch Statement

- Use the switch statement to select one of many blocks of code to be executed.

```
switch(n)
{
  case 1:
    execute code block 1
    break;
  case 2:
    execute code block 2
    break;
  default:
    code to be executed if n is different from case 1 and 2
}
```

The *default* Keyword

- Use the *default* keyword to specify what to do if there is no match.

Conditional Operator

- JavaScript also contains a conditional operator that assigns a value to a variable based on some condition.

```
variablename=(condition)?value1:value2
voteable=(age<18)?"Too young":"Old enough";
```

JavaScript Loops and Repetition

- Loops can execute a block of code a number of times.
- Loops are handy, if you want to run the same code over and over again, each time with a different value.

Different Kinds of Loops

- JavaScript supports different kinds of loops:
 - **for** - loops through a block of code a number of times
 - **for/in** - loops through the properties of an object
 - **while** - loops through a block of code while a specified condition is true
 - **do/while** - also loops through a block of code while a specified condition is true

The For Loop

- The for loop is often the tool you will use when you want to create a loop.
- The for loop has the following syntax:

```
for (statement 1; statement 2; statement 3)
{
  the code block to be executed
}
```

- **Statement 1** is executed before the loop (the code block) starts.
- **Statement 2** defines the condition for running the loop (the code block).
- **Statement 3** is executed each time after the loop (the code block) has been executed.

```
for (var i=0; i<5; i++)
{
  x=x + "The number is " + i + "<br>";
}
```

The For/In Loop

- The JavaScript for/in statement loops through the properties of an object.

```
var person={fname:"John",lname:"Doe",age:25};
for (x in person)
{
  txt=txt + person[x];
}
```

The While Loop

- The while loop loops through a block of code as long as a specified condition is true.

```
while (condition)
{
  code block to be executed
}
```

The Do/While Loop

- The do/while loop is a variant of the while loop. This loop will execute the code block once, before checking if the condition is true, then it will repeat the loop as long as the condition is true.

```
do
{
  code block to be executed
}
while (condition);
```

JavaScript Objects

- JavaScript has several built-in objects, like String, Date, Array, and more.
- An object is just a special kind of data, with **properties** and **methods**.

Accessing Object Properties

- Properties are the values associated with an object.
- The syntax for accessing the property of an object is below

objectName.propertyName

- This example uses the length property of the String object to find the length of a string:

```
var message="Hello World!";  
var x=message.length;
```

Accessing Objects Methods

- Methods are the actions that can be performed on objects.
- You can call a method with the following syntax.

objectName.methodName()

- This example uses the toUpperCase() method of the String object, to convert a text to uppercase.

```
var message="Hello world!";  
var x=message.toUpperCase();
```

Creating JavaScript Objects

- With JavaScript you can define and create your own objects.
- There are 2 different ways to create a new object:
 - Define and create a direct instance of an object.
 - Use a function to define an object, then create new object instances.

Creating a Direct Instance

- This example creates a new instance of an object, and adds four properties to it:

```
person=new Object();  
person.firstname="Narendra";  
person.lastname="Modi";  
person.age=24;  
person.eyecolor="blue";
```

User- Defined Objects/How user defined objects are created in JavaScript?

- JavaScript allows you to create your own objects.